Claude Code Telegram Bot Fleet

Operations, monitoring & reproduction guide

Machine: this Windows 11 desktop · Generated 2026-06-22 · Owner: Bruno Dias (bruno.dias@secil.pt)
Scripts: `D:\Github\BD\Agents\claude-telegram\` · State: `~/.claude/channels/<bot>/`

1. What this is

Three Telegram bots run as **Claude Code channels** — an inbound Telegram message enters a local `claude --channels` session, which can read repo files and call MCP servers, then replies back through the bot. A fourth bot (**OpenClaw**) is a *separate runtime* and is documented here only for awareness.

Plugin	`telegram@claude-plugins-official` v0.0.6 (MCP server on Bun)
Launch	One Windows Terminal window, one titled tab per bot, each under a restart supervisor
Autostart	Scheduled task `\BD\Bots\ClaudeTgFleet` (at logon +30s)
Monitoring	Scheduled task `\BD\Bots\ClaudeTgFleetMonitor` (every 5 min, toasts on problems)

2. The fleet

Bot (tab)	Telegram	Token	Model	Repo root (context)	State dir
uns-ot-expert tg:uns-ot-expert	@uns_ot_expert_bot	8729…	default (CC built-in)	`D:\Github\SECIL\CodingAgentIgnition`	`~/.claude/channels/uns-ot-expert`
bd-claude-vmhost1 tg:bd-vmhost1	@bd_claudevmhost1_bot	8637…	claude-sonnet-4-6	`D:\Github\BD`	`~/.claude/channels/bd-claude-vmhost1`
generic-channel tg:bd-secil-claude	@BDSecilClaudeBot ("BD@SECIL | CLAUDE v0")	8720…	claude-sonnet-4-6	`D:\Github\BD\Agents\claude-telegram\generic-channel`	`~/.claude/channels/generic-channel`
OpenClaw — separate gateway runtime at `C:\Users\bsdias\.openclaw`, token 8215…. Not managed by these scripts. See §9.					

3. The problem we fixed

The telegram plugin keeps its token, allowlist (`access.json`) and single-instance lock (`bot.pid`) in **one shared state directory** — by default `~/.claude/channels/telegram` . Every `claude --channels` session (and in fact *any* plain claude session that loads the plugin) used that one directory, so on each start a new instance **SIGTERMed the previous poller** (Telegram allows only one `getUpdates` consumer per token). Result: only one bot was ever alive; uns-ot-expert sat dead with unconsumed updates.

> **Fix — one isolated state dir per bot.** Each bot now launches with `TELEGRAM_STATE_DIR = ~/.claude/channels/<bot>` , so token, allowlist and pid-lock are independent and all bots run concurrently. The plugin's own docs prescribe exactly this for multi-bot machines. The shared global `.env` was made **tokenless** so stray plain sessions no longer fight a real bot for its token.

4. Day-to-day operations

All commands run from `D:\Github\BD\Agents\claude-telegram\` .

Goal	Command

Health of every bot	`.\Channels.ps1 doctor`
Start the whole fleet (titled tabs)	`.\Channels.ps1 start-all`
Start / restart one bot	`.\Channels.ps1 start -Name uns-ot-expert` · `restart -Name …`
Stop one / all	`.\Channels.ps1 stop -Name …` · `stop-all`
Tail a bot's supervisor log	`.\Channels.ps1 logs -Name uns-ot-expert -Follow`
Tail plugin (MCP) stderr	`.\Channels.ps1 logs -Name uns-ot-expert -Mcp -Follow`
Show allowlists & pending pairings	`.\Channels.ps1 ids`
Add a device/user to a bot	`.\Channels.ps1 allow -Name uns-ot-expert -Value <numericId>`
Set access policy	`.\Channels.ps1 policy -Name uns-ot-expert -Value allowlist`

The **tab title** is the bot name (e.g. `tg:uns-ot-expert`) so tabs are easy to tell apart. Each tab shows the live session and, with `--debug`, the plugin's stderr. Closing a tab stops that bot (re-open with `start -Name`).

5. Logs & debugging

What	Where
Supervisor lifecycle (start / exit code / restart)	`~/.claude/channels/<bot>/logs/supervisor-YYYYMMDD.log`
Per-bot status snapshot	`~/.claude/channels/<bot>/status.json`
Plugin stderr (polling / 409 / handler errors)	`%LOCALAPPDATA%\claude-cli-nodejs\Cache\<repo-slug>\mcp-logs-plugin-telegram-telegram\*.jsonl`
Live token-side truth	`getWebhookInfo` — see `pending_update_count` & `last_error_message`

"My bot is silent" — decision tree

1. **Token?** `.\Channels.ps1 doctor` → STOPPED "no token" → fix the bot's `.env`.
2. **Poller alive?** doctor "POLLER = dead" → supervisor will restart; check the supervisor log for crash-loop (bad model id, missing Bun).
3. **Updates piling up?** `pending` climbing > 0 → nothing is consuming → a duplicate poller elsewhere (look for `409 Conflict` in `-Mcp` log).
4. **Sender allowed?** A silent drop on a real message = sender not in this bot's `access.json` allowlist (policy `allowlist` drops strangers silently). `.\Channels.ps1 allow …`.
5. **Bun?** `bun --version` must work; the MCP server runs on Bun.

6. Always-on monitoring

You know the fleet state three ways, in increasing automation:

- **On demand:** `.\Channels.ps1 doctor` — colored table: OK / WARN / STOPPED.
- **Continuous file:** the monitor task writes `~/.claude/channels/fleet-status.json` every 5 minutes.
- **Push:** if any bot is STOPPED or erroring, the monitor raises a **Windows toast** and appends to `~/.claude/channels/monitor.log`.

A bot is WARN if Telegram reports a recent `last_error` or > 5 unconsumed updates; STOPPED if its token is missing/invalid or no live poller holds `bot.pid`. Verified resilience: killing a session triggers an automatic restart in ~2 s.

7. Access policy

A Telegram **numeric user ID identifies an account, not a device** — your phone and laptop signed into the same account share one ID; only a separately-registered account (e.g. a userbot on the server) is a different ID.

Policy	Behaviour	Use
`pairing`	Reply with a 6-char code, drop the message until approved	Bootstrap only — to capture unknown IDs
`allowlist`	Silently drop anyone not on the list	**Target end-state** for a public bot
`disabled`	Drop everything	Pause a bot without stopping it

Capture your phone / laptop / server IDs and lock down

1. From each account, DM @userinfobot → it replies with the numeric ID. (Phone+laptop on the same account = same ID.)
2. Add each to every bot you want it to reach: `.\Channels.ps1 allow -Name <bot> -Value <id>` (applied live, no restart).
3. Lock each bot: `.\Channels.ps1 policy -Name <bot> -Value allowlist`.

Alternatively, with policy `pairing`, DM the bot itself; the code/ID lands in the bot's `access.json` "pending" (see `.\Channels.ps1 ids`), then `allow` it.

8. Recreate the fleet from scratch / deploy elsewhere

Everything needed to reproduce this on a new machine or for a brand-new bot.

8.1 Prerequisites

```
# Claude Code v2.1.80+ signed in with a claude.ai (Pro/Max) account — NOT api-key
claude --version
# Bun runtime (the plugin's MCP server runs on Bun)
bun --version            # install: npm i -g bun   (or https://bun.sh)
# Install the channel plugin once, then reload
#   /plugin install telegram@claude-plugins-official
#   /reload-plugins
```

8.2 Create a bot in Telegram

1. DM **@BotFather** → `/newbot` → set a name and a username ending in `bot`.
2. Copy the token (looks like `1234567890:AAH…`).
3. Optional: `/setdescription`, `/setuserpic`; for DM-only leave `/setprivacy` Enabled.

8.3 Create the profile

```
# A profile is a folder with a .env (token) and optional SYSTEM_PROMPT.md (persona)
mkdir <profileDir>
#   <profileDir>\.env            →  TELEGRAM_BOT_TOKEN=1234567890:AAH...
#   <profileDir>\SYSTEM_PROMPT.md→  the bot's persona (appended via --append-system-prompt)
#   <profileDir>\.gitignore      →  .env
```

8.4 Register it in the manifest

Add an entry to `D:\Github\BD\Agents\claude-telegram\channels.json`:

```
{
  "name": "my-bot",
  "profileDir": "D:\\path\\to\\profile",
  "repoRoot":   "D:\\repo\\to\\load\\as\\context",
  "model":      "claude-sonnet-4-6",
  "tabTitle":   "tg:my-bot"
}
```

The bot `name` is also its state-dir name under `~/.claude/channels/`. Keep names unique. Give each bot a **distinct** `repoRoot` so their plugin (MCP) logs land in separate folders.

8.5 Seed access and run

```
# seed your own id and start locked-down (or pairing to capture others)
.\Channels.ps1 allow  -Name my-bot -Value <yourNumericId>
.\Channels.ps1 policy -Name my-bot -Value allowlist
.\Channels.ps1 start  -Name my-bot      # or start-all for the whole fleet
.\Channels.ps1 doctor
```

8.6 Autostart + monitoring (Windows)

```
# Logon task → opens the wt window with all tabs
Register-ScheduledTask -TaskName 'ClaudeTgFleet' -TaskPath '\BD\Bots\' `
  -Action (New-ScheduledTaskAction -Execute (Get-Command pwsh).Source `
     -Argument '-NoProfile -ExecutionPolicy Bypass -File "D:\Github\BD\Agents\claude-telegram\Channels.ps1" start-
all') `
  -Trigger (New-ScheduledTaskTrigger -AtLogOn) `
```

```
  -Principal (New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME" -LogonType Interactive)
# Monitor task → every 5 min: fleet-status.json + toast on problems  (Channels.ps1 monitor)
```

8.7 Deploy on a Linux server

The plugin is cross-platform; only the supervisor/title layer is Windows-specific. On Linux:

```
# per bot, in its own tmux window (named = the bot):
export TELEGRAM_STATE_DIR="$HOME/.claude/channels/my-bot"
export TELEGRAM_BOT_TOKEN="1234567890:AAH..."
cd /repo/to/load
tmux new -d -s tg-mybot \
  'while true; do claude --channels plugin:telegram@claude-plugins-official \
      --model claude-sonnet-4-6 --append-system-prompt "@/path/SYSTEM_PROMPT.md" --debug \
      2>&1 | tee -a "$TELEGRAM_STATE_DIR/logs/session.log"; sleep 2; done'
# Or a systemd --user service per bot with Restart=always and the same env vars.
# access.json lives at $TELEGRAM_STATE_DIR/access.json (edit to add allowFrom ids).
```

> **Golden rule for any number of bots on one host:** one `TELEGRAM_STATE_DIR` per bot, never shared. That single setting is what keeps their tokens, allowlists and pollers from colliding.

9. OpenClaw — advisory (not changed)

OpenClaw is a separate gateway runtime (own token 8215…, own pairing store at `~/.openclaw/credentials/`); it does **not** share the plugin's state and never collided with the three bots. No settings were changed. Observations:

- Its `dmPolicy` is `pairing` on a publicly-addressable bot — it answers strangers. Consider `allowlist` (its `groupPolicy` is already `allowlist`).
- A stranger pairing request is queued (code `GXD52427`, name "雨舞霓裳", 2026-05-29) — evidence the bot is being probed. Worth clearing.
- Token is distinct from the three managed bots — no Telegram-side conflict.

10. File reference

Path	Role
`…\claude-telegram\channels.json`	Manifest — the single source of truth (name, profile, repo, model, title)
`…\claude-telegram\Start-Channel.ps1`	Launch ONE bot in the foreground with its isolated state dir
`…\claude-telegram\Watch-Channel.ps1`	Supervisor — title, restart loop, status.json, supervisor log
`…\claude-telegram\Channels.ps1`	Management CLI — doctor/logs/start/stop/restart/monitor/ids/allow/policy
`<profileDir>\.env` · `SYSTEM_PROMPT.md`	Per-bot token + persona
`~/.claude/channels/<bot>/`	Per-bot state: access.json, bot.pid, status.json, logs/
`~/.claude/channels/telegram/.env`	Global plugin state — intentionally tokenless

Appendix — Secrets & IDs (PRIVATE — your eyes only)

> ⚠ **CONFIDENTIAL.** This page contains live bot tokens and account IDs in clear text, at your explicit request, for personal reference. Anyone with a bot token has full control of that bot. **Do not share, print, email, or commit this PDF.** Tokens are *not* stored in git — only in each bot's gitignored `.env` and here. If this file leaks, rotate every token via @BotFather (`/revoke`) and re-issue.

Bot tokens (full)

Bot	Telegram username	Bot numeric ID	Token (TELEGRAM_BOT_TOKEN)
uns-ot-expert	@uns_ot_expert_bot	8729025603	`8729025603:AAH5MofA_t09c_4GE4GbfLSywuWlTzGusX0`
bd-claude-vmhost1	@bd_claudevmhost1_bot	8637190243	`8637190243:AAHy32Z3tQDNdcfGBT3gnjt3h6wCjjfL664`
generic-channel	@BDSecilClaudeBot	8720091908	`8720091908:AAFf3_LAN-mKO6y3Yj2FSIMH6x9T4EAVQ3g`
OpenClaw (separate runtime)	@BDOpenClaw_secilbot	8215764825	`8215764825:AAGL_t7bjvKVYhIrF89Q-qR4Ln5UG3j4iqo`

Token files: `<profileDir>\.env` per bot. OpenClaw token lives in `~/.openclaw/openclaw.json` → `telegram.botToken`.

Account / user IDs (allowlists)

Numeric ID	Who	Allowed on	Note
1389916364	**You** (main Telegram account — phone & laptop share this)	all 3 bots + OpenClaw	Pre-seeded everywhere
8714497602	Second account — **identity TBC**	bd-claude-vmhost1	Inherited from the old shared allowlist; confirm or remove
5321515964	"雨舞霓裳" — **unknown stranger**	OpenClaw (pending, NOT approved)	Spam pairing probe — deny it

A Telegram numeric ID = an **account**, not a device. To capture a new one: DM @userinfobot from that account, then `.\Channels.ps1 allow -Name <bot> -Value <id>`.

Other secrets

OpenClaw gateway token	`445e93ca4c9e512b6c1471639b9f8cd693598c16827094df` (~/.openclaw/openclaw.json, mode=token — OpenClaw-internal, not a Telegram token)

Quick reference — what each bot is for

Bot	Role	Model
@uns_ot_expert_bot	UNS/OT corporate Q&A (Canary, EMQX, Ignition, SSH)	default (Claude Code built-in)
@bd_claudevmhost1_bot	Personal assistant	claude-sonnet-4-6
@BDSecilClaudeBot	BD@SECIL general assistant	claude-sonnet-4-6
@BDOpenClaw_secilbot	OpenClaw gateway (separate product)	per OpenClaw config

Generated for Bruno Dias · Claude Code · 2026-06-22. **Private document** — contains live credentials by request.